Prueba técnica – Analytics Engineer Junior

Informe de respuestas

Humberto Alexander de la Cruz

5 de octubre de 2026

1. Alcance y herramientas

La solución se desarrolló en Databricks Free Edition con compute serverless, Unity Catalog y tablas Delta. Python se usó para la ingesta y la limpieza (PySpark) y Databricks SQL para las consultas. Los datos se organizan con arquitectura medallion:

- **Bronze:** datos tal como llegan. Los CSV se cargan con todas las columnas como texto y la respuesta del Banguat se guarda como XML sin modificar.
- **Silver:** datos con tipos correctos, sin duplicados, normalizados y con indicadores de calidad.
- **Gold:** modelo dimensional para análisis.

Los notebooks se ejecutan en orden con un Job de Databricks de seis tareas (sección 8).

2. Supuestos

Sobre el enunciado

- Las referencias al "ejercicio 5" (ejercicio independiente, tabla `saldos` y archivo de corrección) corresponden al ejercicio 4 de depuración.
- La tabla `tipo_cambio` alimenta los ejercicios 2 y 3; la depuración no la usa.
- "El modelo del ejercicio 4" se refiere a un ejercicio de modelado que no aparece en el enunciado. Se entrega un modelo estrella como capa gold (sección 7).
- La pregunta "Responde brevemente: en producción" está incompleta. Se responde cómo se operaría el proceso en producción.

Sobre los datos

- Las fechas con barras (`15/02/2025`) están en formato día/mes/año.
- `fecha_hora` está en hora de Guatemala, la misma referencia que la fecha del tipo de cambio.
- Un monto negativo en un débito es una convención de signo: la dirección del movimiento la indica `tipo`.
- En el ejercicio 3 solo se consideran transacciones `COMPLETADA`, y el monto total transaccionado es la suma de créditos y débitos (volumen), no el neto.

3. Ejercicio 1: tipo de cambio del Banco de Guatemala

3.1. Método del WSDL

El servicio expone las siguientes operaciones:

Operación	Parámetros
`TipoCambioDia`, `TipoCambioDiaString`	Ninguno (tasa del día)
`TipoCambioFechaInicial`	`fechainit`
`TipoCambioFechaInicialMoneda`	`fechainit`, `moneda`
`TipoCambioRango`	**`fechainit`, `fechafin`**
`TipoCambioRangoMoneda`	`fechainit`, `fechafin`, `moneda`
`Variables`, `VariablesDisponibles`	`variable` / ninguno

Tabla 1: Operaciones del servicio `TipoCambio.asmx`.

El método para un rango de fechas es `TipoCambioRango`. Recibe `fechainit` y `fechafin` como texto en formato `dd/mm/yyyy` y responde una lista `Vars/Var` con `moneda`, `fecha`, `venta` y `compra`, además de `TotalItems`.

3.2. Implementación

La función `obtener_tipo_cambio(fecha_ini, fecha_fin)` de `src/banguat.py` arma la petición SOAP, la envía y devuelve una tabla con `fecha` y `tasa`. El rango se calcula a partir de `transacciones.csv` (del 01/06/2025 al 10/06/2025) y se consulta en una sola llamada.

- **Bronze** (`bronze.tipo_cambio_raw`): una fila por consulta con el XML crudo, la fecha y hora de la consulta, el código HTTP, el número de intentos y el error, si lo hubo.
- **Silver** (`silver.tipo_cambio`): columnas `fecha` y `tasa`, cargadas con `MERGE` por fecha para que el proceso se pueda volver a ejecutar sin duplicar.

Fecha	Tasa	Fecha	Tasa
2025-06-01	7.67644	2025-06-06	7.68243
2025-06-02	7.67568	2025-06-07	7.68243
2025-06-03	7.67867	2025-06-08	7.68243
2025-06-04	7.67883	2025-06-09	7.68288
2025-06-05	7.68322	2025-06-10	7.68286

Tabla 2: Tabla `tipo_cambio` (quetzales por dólar).

Observaciones del servicio:

- Devuelve todos los días del calendario: sábado y domingo repiten la tasa del viernes, por lo que cada transacción tiene la tasa de su propio día.
- `venta` y `compra` traen el mismo valor (tipo de cambio de referencia). Se usa `venta`.
- Si las fechas no se envían en `dd/mm/yyyy`, el servicio no devuelve error: con `2025-06-01` respondió datos desde el 6 de enero. Por eso se valida que las fechas recibidas estén dentro del rango pedido.
- Un rango sin datos responde HTTP 200 con `TotalItems = 0`.

3.3. Manejo de errores

Situación	Comportamiento
Timeout, error de conexión, HTTP 5xx o 429	Hasta 3 intentos con espera exponencial; después `ServicioNoDisponibleError`
HTTP 4xx	`RespuestaHTTPError`, sin reintento
XML inválido o SOAP Fault	`XMLInvalidoError`
Respuesta sin tasas	`SinDatosError`
Fechas fuera del rango pedido	`RespuestaInconsistenteError`
Tasa fuera de 5–12 quetzales por dólar	No se escribe silver

Tabla 3: Manejo de errores del cliente del Banguat.

Las consultas fallidas también se registran en bronze. El notebook incluye una prueba de cada caso: timeout, endpoint inexistente (el servidor responde HTTP 500), XML inválido y rango sin datos.

3.4. Servicio de respaldo

El Banguat fue accesible desde Databricks (HTTP 200), por lo que no fue necesario usar `open.er-api.com`. Ese servicio solo entrega la tasa del día en que se consulta: las transacciones de junio de 2025 se habrían convertido con la tasa actual y no con la oficial de su fecha, además de no ser una fuente oficial.

3.5. En producción

- Un Job diario que encadena la ingesta del tipo de cambio, la limpieza y el modelo, con alertas si falla.
- Carga incremental desde la última fecha cargada, con algunos días de traslape por posibles correcciones del Banguat, y `MERGE` por fecha.
- Bronze como registro de auditoría de cada respuesta del servicio.
- Validación de días faltantes, rango de fechas y tasas plausibles antes de publicar silver.
- URL, tiempos de espera y catálogo como parámetros del Job.
- Si el servicio no responde, la corrida se marca como fallida y se reintenta; no se usa la tasa de otra fecha.

4. Ejercicio 2: limpieza e integración con el tipo de cambio

4.1. Problemas detectados y decisiones

Solo se eliminan duplicados exactos. Los demás problemas se corrigen con una regla explícita o se marcan con un indicador, y quedan contados en la tabla `silver.dq_reporte`.

4.2. Transacciones en dólares sin tasa

Se marcan con `flag_usd_sin_tasa` y su `monto_gtq` queda NULL, de modo que no se suman con un valor incorrecto. En esta carga no hubo casos, porque el Banguat devolvió tasa para todos los días del rango. Si ocurriera, se volvería a consultar el servicio para esas fechas en la siguiente

Problema	Registros	Decisión	Motivo
Duplicados exactos	Cliente 2, T004	Se elimina la copia	Contarlos dos veces infla montos y conteos
`fecha_alta` en `dd/MM/yyyy`	Cliente 2	Se convierte a DATE	Formato local; no hay fechas ambiguas
Departamento con formato distinto	Cliente 2	`initcap(trim())` y validación contra los 22 departamentos	Con "`guatemala`" en minúscula y con espacio quedaría como otro departamento
Moneda en minúscula	T007	`upper(trim())`	Sin esto no encuentra tasa
Segmento vacío	Cliente 4	Se asigna `No definido`	Valor desconocido explícito en lugar de NULL
Monto nulo	T006	`flag_monto_nulo`; `monto_gtq` NULL	No se puede inventar el monto; queda fuera de las sumas pero visible
Monto negativo	T003	`flag_monto_negativo`; `monto_gtq` con valor absoluto	La dirección ya la indica `tipo`; se conserva el monto original
Cuenta con cliente inexistente	Cuenta 106 (cliente 9)	Se conserva con `flag_cliente_inexistente`	Eliminarla ocultaría que falta el cliente en la extracción

Tabla 4: Problemas de calidad y decisiones.

corrida; el `MERGE` agregaría la tasa y la transacción se recalcularía. Usar la última tasa disponible anterior solo se haría con aprobación del negocio y dejando la transacción marcada.

4.3. Resultado: `transacciones_gtq`

La conversión es `monto_gtq` = |`monto`| × `tasa`, con tasa 1 para quetzales, redondeada a centavos por transacción.

ID	Tipo	Estado	Monto	Moneda / tasa	`monto_gtq`	Indicador
T001	Crédito	Completada	500.00	GTQ	500.00	
T002	Débito	Completada	120.50	USD 7.67568	924.92	
T003	Débito	Completada	−80.00	GTQ	80.00	Monto negativo
T004	Crédito	Completada	1,500.00	GTQ	1,500.00	
T005	Débito	Rechazada	200.00	USD 7.67883	1,535.77	
T006	Débito	Completada	NULL	GTQ	NULL	Monto nulo
T007	Débito	Completada	75.25	USD 7.68286	578.14	

Tabla 5: Transacciones en quetzales.

5. Ejercicio 3: consultas SQL

Motor: Databricks SQL (Spark SQL sobre tablas Delta). Las consultas están en `sql/ejercicio3_ejercicio`

3.1 Monto total de créditos y débitos por cliente

Luis Ramírez tiene una transacción completada (T006), pero sin monto.

Cliente	Créditos (GTQ)	Débitos (GTQ)
1 Ana López	500.00	1,583.06
2 Carlos Pérez	1,500.00	0.00
3 María Gómez	0.00	0.00
4 Luis Ramírez	0.00	0.00
5 Sofía Castillo	0.00	0.00

Tabla 6: Créditos y débitos por cliente.

3.2 Clientes sin transacciones

María Gómez, cuya única cuenta está cerrada y sin movimientos, y Sofía Castillo, que no tiene cuentas.

3.3 Cliente con mayor monto por departamento

Se usa `RANK()` particionado por departamento, para que en caso de empate aparezcan todos los clientes empatados.

Departamento	Cliente	Monto total (GTQ)
Escuintla	4 Luis Ramírez	0.00
Guatemala	1 Ana López	2,083.06
Quetzaltenango	3 María Gómez	0.00
Sacatepéquez	5 Sofía Castillo	0.00

Tabla 7: Cliente con mayor monto transaccionado por departamento.

Guatemala es el único departamento con más de un cliente una vez normalizado el texto: Ana López (2,083.06) supera a Carlos Pérez (1,500.00). Sin la normalización, Carlos Pérez quedaría solo en un departamento "guatemala".

6. Ejercicio 4: depuración

6.1. Por qué el saldo sale inflado

`saldos` y `transacciones` tienen varias filas por cuenta y la consulta las une a `cuentas` al mismo tiempo. Para cada cuenta se generan $n_{\text{saldos}} \times n_{\text{transacciones}}$ filas: cada saldo se suma una vez por cada transacción de la cuenta y cada transacción se cuenta una vez por cada saldo.

Además, el saldo es una foto por fecha: se puede sumar entre cuentas, pero no a lo largo del tiempo. Aunque no hubiera transacciones, sumar todas las fechas de `saldos` daría un valor sin sentido. Por último, el `INNER JOIN` con `transacciones` excluye las cuentas sin movimientos.

6.2. Consulta corregida

Cada tabla de detalle se agrega a nivel de cuenta antes del join (último saldo y número de transacciones por cuenta), y se une con `LEFT JOIN`.

```
WITH saldo_actual AS (
    SELECT cuenta_id, saldo
    FROM (
        SELECT cuenta_id, saldo,
```

```
            ROW_NUMBER() OVER (PARTITION BY cuenta_id ORDER BY fecha DESC) AS rn
        FROM saldos
    ) s
    WHERE rn = 1
),
transacciones_por_cuenta AS (
    SELECT cuenta_id, COUNT(*) AS num_transacciones
    FROM transacciones
    GROUP BY cuenta_id
)
SELECT
    c.segmento,
    SUM(s.saldo) AS saldo_total,
    COALESCE(SUM(t.num_transacciones), 0) AS num_transacciones
FROM clientes c
JOIN cuentas cu ON c.cliente_id = cu.cliente_id
LEFT JOIN saldo_actual s ON cu.cuenta_id = s.cuenta_id
LEFT JOIN transacciones_por_cuenta t ON cu.cuenta_id = t.cuenta_id
WHERE c.segmento = 'Premium'
GROUP BY c.segmento;
```

Como no se entregaron datos de `saldos`, se probó con una tabla de ejemplo de tres fechas por cuenta para el cliente Premium (cuentas 101 y 102):

Consulta	Saldo total	Transacciones
Original	11,250.00	12
Corregida	1,600.00	4

Tabla 8: Comparación con datos de ejemplo. La cuenta 101 genera $3 \times 3 = 9$ filas y la 102, $3 \times 1 = 3$.

7. Modelo dimensional

Tabla	Grano	Columnas principales
`fact_transacciones`	Una transacción	`transaccion_id`, `fecha_key`, `cuenta_id`, `cliente_id`, `tipo`, `canal`, `estado`, `moneda`, `tasa_aplicada`, `monto_gtq`, `monto_gtq_neto`
`dim_cliente`	Un cliente	`cliente_id`, `nombre`, `departamento`, `segmento`, `fecha_alta`
`dim_cuenta`	Una cuenta	`cuenta_id`, `cliente_id`, `tipo_cuenta`, `fecha_apertura`, `estado`
`dim_fecha`	Un día	`fecha_key`, `fecha`, `anio`, `mes`, `nombre_mes`, `nombre_dia`, `es_fin_de_semana`

Tabla 9: Modelo estrella de la capa gold.

- `monto_gtq` es el volumen y `monto_gtq_neto` suma créditos y resta débitos.
- La fact incluye la llave del cliente además de la de la cuenta, para analizar por cliente sin pasar por `dim_cuenta`.

- `dim_cliente` tiene un registro `-1 Desconocido` para cuentas cuyo cliente no existe.
- `tipo`, `canal`, `estado` y `moneda` quedan en la fact porque tienen pocos valores.
- `saldos` se modelaría como una fact aparte de foto periódica (grano cuenta por día), lo que evita el problema del ejercicio 4.

La carga produjo 6 clientes (incluido el desconocido), 6 cuentas, 147 días y 7 transacciones. El monto total en quetzales de gold (5,118.83) coincide con el de silver.

8. Ejecución

Los notebooks se ejecutan con el Job `pipeline_prueba_analytics`: seis tareas encadenadas (setup, bronze, ejercicios 1 a 4 y modelo gold) con compute serverless. La ejecución terminó correctamente en 6 minutos y 2 segundos.